15,471,856 members Sign in

articles Q&A forums stuff lounge ? Search for articles, questions,

DotNetMQ: A Complete Message Queue System for .NET



Halil ibrahim Kalkan

23 May 2011 LGPL3

Rate me: 4.94/5 (189 votes)

A new and independent Open Source Message Queue system that is entirely built in C# and .NET framework 3.5.

Download source code - 1.28 MB

Download binaries - 933 KB

Download samples - 534 KB

Article Outline

- Introduction
- What is **Messaging**?
- What is **DotNetMQ**?
- Why a New **Message Broker**?
  - The **Need** for a Message Broker
  - What About **existing** Message Brokers
- **Installing** and Running DotNetMQ
- **First Application** Using DotNetMQ
  - **Registering** Applications to DotNetMQ
  - Developing **Application1**
  - Developing **Application2**

    - **Transmit Rule** Property of Message
    - **CommunicationWay** Property of MDSClient
    - **ReConnectServerOnError** Property of MDSClient
    - **AutoAcknowledgeMessages** Property of MDSClient
- **Configuring** DotNetMQ
    - **Servers**
    - **Applications**
    - **Routing** / **Load Balancing**
    - **Other Settings**
- Messaging Over **Network**
    - A Simple Application
    - A Real Life Case: **Distributed SMS Processor**
- **Request/Reply** Style Messaging
- **Service-Oriented** Architecture on DotNetMQ
    - Sample Application: **SMS/Mail Sender**
        - Service
        - Client
    - **Web Services** Support
- **Performance** of DotNetMQ
- **History**
- **References**

Introduction

In this article, I will introduce a new and independent **Open Source Message Queue system** that is entirely built in C# and .NET framework 3.5. **DotNetMQ** is a message broker that has several features including guaranteed delivering, routing, load balancing, server graphs... so on. I will start by explaining messaging concepts and the need for message brokers. Then I will examine what DotNetMQ is and how to use it.

What Is Messaging?

Messaging is a way of ***asynchronous*** communication of applications running on same or different machines with reliable delivery. Programs communicate by sending packets of data called messages to each other [1].

A message may be a string, a byte array, an object... etc. Typically, a **sender (producer)** program creates a message and pushes it to a message queue and a **receiver (consumer)** program gets the

message from the queue and processes it. The sender and receiver programs donâ€™t have to be running at the same time, since messaging is an asynchronous process. This is called **loosely coupled** communication.

On the other hand, a Web Service method call (Remote Method Invocation) is a type of **tightly coupled** and **synchronous** communication (both applications have to be running and available during the whole communication; if the Web Service is offline or an error occurs during the method call, the client application gets an exception).

Figure - 1: Simplest messaging of two applications.

In the figure above, two applications communicate over a message queue in a loosely coupled manner. If the receiver consumes messages slower than the sender produces it, the message count on the queue will increase. Also, the receiver may be offline while the sender is sending messages. In this situation, the receiver gets the messages from the queue when it becomes online (when it starts and joins the queue).

Message Queues are typically provided by Message Brokers. A **Message Broker** is a standalone application (service) that other applications connect to and send/receive messages. A Message Broker is responsible to store messages until a receiver receives them. A Message Broker can route messages across machines to deliver a message to the destination application and can try delivering the message until the receiver correctly handles it. A Message Broker is sometimes called a **Message Oriented Middleware** (**MOM**) or simply **Message Queue** (**MQ**).

What is DotNetMQ?

DotNetMQ is an open source Message Broker that has several features:

- **Persistent** or **non-persistent** messaging.
- **Guaranteed delivery** of persistent messages even in a system crash.
- **Automatic** and **manual routing** of messages in a custom **machine graph**.
- Supports **multiple databases** (**MS SQL Server, MySQL**, **SQLite**, and memory-based storage for now).
- Supports **donâ€™t store, direct send** style messaging.
- Supports **Request/Reply** style messaging.
- **Easy to use** client library to communicate with the DotNetMQ Message Broker.
- Built-in framework to easily construct **RMI services** upon message queues.
- Supports **delivering messages to ASP.NET Web Services**.
- GUI-based **management** and **monitoring** tool.
- Easy to install, manage, and use.

- Written entirely in **C#** (using .NET Framework 3.5).

I preferred to name DotNetMQ as MDS (Message Delivery System) when first creating it. Because it is designed not just to be a message queue, but also as a system that delivers messages directly to applications and an environment that provides a framework to build application services. I called it **DotNetMQ** since it is entirely developed using .NET and the DotNetMQ name is more memorable. So, itâ€™s original name (and internal project name) is MDS and the applications have many classes with the prefix **MDS**.

Why a New Message Broker?

The Need for a Message Broker

First, I will demonstrate a simple situation where a message broker is needed.

In my experiences in business life, I've observed really bad and uncommon asynchronous enterprise application integration solutions. Usually there is an application that runs on a server and performs some tasks and produces data, and then sends the result data to another application on another server. The second application performs other tasks on the data or evaluates the result (the servers are on the same network or connected over the internet). Also, the message data must be persistent. Even if the remote application is not working or the network is not available, the **message must be delivered on the first chance**.

Letâ€™s look at the design in the figure below.

Figure - 2: A bad solution to integrate applications.

Application - 1 and **Application - 2** are executable applications (or Windows services) and **Sender Service** is a Windows service. Application - 1 performs some task, produces data, and calls a **Remote Web Service** method on **Server - B** to transmit data. This Web Service inserts data into a **database table**. Application - 2 periodically checks the table for new incoming data rows and processes them (and deletes them from the table or marks them as processed to not process the same data again).

If an **error** occurs during the Web Service call or while processing data in the Web Service, data must not be lost and must be sent later. However, Application - 1 has other tasks to do, so it can not try to send data again and again. It simply inserts data into a database table. Another Windows service (or a thread in Application - 1, if the application always runs) checks this table periodically and tries to send data to the Web Service until data is successfully sent.

This scenario is really reliable (messages are guaranteed to be delivered) but is not an efficient way of communicating between two applications. This solution has some very critical problems:

- It takes a **long time to develop** (to code).
- Individual coding for all **message types** (or remote method calls). For a new Web Service method call, you must change all the services, applications, and database tables.
- Almost same software and structures must be developed (or copied and modified) for every similar service.
- Testing and maintenance of **too many services/applications/databases** after coding.
- Some applications and services periodically check the database even if there is no new message (if the database is not well indexed and optimized, this may consume serious system resources).

Message Brokers do all this job and takes all the responsibility to deliver messages to the remote application in the most efficient way. The same application integration using DotNetMQ is shown in the figure below.

Figure - 3: Simple messaging by using DotNetMQ.

DotNetMQ is a standalone Windows service that runs on both **Server - A** and **Server - B**. Thus, you just need to write code to communicate with DotNetMQ. Using the DotNetMQ Client Library, it is very easy and fast to connect and send/receive messages to/from the DotNetMQ service. **Application - 1**

prepares the message, sets the destination, and passes the message to the DotNetMQ Broker. DotNetMQ brokers will deliver the message to **Application - 2** in the most efficient and fastest way.

What About Existing Message Brokers

It is clear to see that there is a need for Message Brokers to integrate applications. I searched the web, and read books to find a free (and Open Source, if available) Message Broker that is easy to use with .NET. Letâ€™s talk about what I found:

- **Apache ActiveMQ** (http://activemq.apache.org): It is Open Source and implements **JMS** (Java Message Service is a standard API for messaging in the Java world). It has also a .NET client library. I read a complete book "ActiveMQ in Action" to learn more and I developed some simple applications. Even though I read the book, I did not see an easy and reliable way to construct an ActiveMQ **server graph** that worked together and routed messages. I also did not see a way to set the destination server for a message. It routes messages automatically but I can not control the routing efficiently. I understood that it is commonly used with **Apache Camel** (http://camel.apache.org) to achieve common application integration patterns. Apache Camel is also another world to discover, and even worse, it is just for Java. Finally, I think that it is not simple enough to use and especially to configure, monitor, and manage it. So I gave up working on ActiveMQ.
- **MSMQ** (http://msdn.microsoft.com/en-us/library/ms711472(VS.85).aspx): This is a solution from **Microsoft** and it is the most suitable framework to use with .NET applications. It is easy to use and learn, and it has tools to monitor queues and messages. It is especially very suitable for asynchronous communication of applications that are running on the same machine or can directly connect to the same machine. But I could not find a built-in solution to construct a graph of MSMQ servers that route messages. Since routing is my first start point, I eliminated this Broker.
- **RabbitMQ** (http://www.rabbitmq.com): It is developed using the **Erlang** programming platform (that is developed by **Ericsson**). You need to install Erlang first. I spent a lot of time to install, configure, and write a sample application. It has a .NET client but I got many errors when trying to develop and run a simple application. It was very hard to install and to make two rabbitMQ Brokers work together on two different servers. After a few days, I gave up because I thought it must not be that hard to learn and to start developing applications.
- **OpenAMQ** (http://www.openamq.org), **ZeroMQ** (http://www.zeromq.org): **I examined these brokers overall** but I found that I can not easily do what I want to using .NET.
- **Others**: I also found a few other projects but they have important features missing like routing, persistent messaging, request/reply messaging... etc.

You see that there is no Message Broker that is developed entirely in .NET in the list above.

From a user perspective, I just want to pass "**message data, destination server, and application name**" to my **local** Broker. I am not interested in the rest. It will route a message over the network as many times it requires and delivers the message to my destination application on the destination

server. My messaging system must provide this simplicity for me. This was my first start point and I evaluated Message Brokers according to that point. The figure below shows what I want.

Figure - 4: Automatic routing messages in a Message Broker servers graph.

Application - 1 passes a message to **Message Broker** in the local server (***Server - A***):

- Destination server: **Server - D**
- Destination application: **Application - 2**
- Message Data: ***application specific data***

Server - A has no direct connection to **Server - D**. So the Message Brokers forward the message over the servers (the message is transmitted through Server - A, Server - B, Server - C, and Server - D sequentially) and the message finally reaches the Message Broker in Server - D to deliver the message to **Application - 2**. Note that there is another instance of Application - 2 running on Server - E, but it does not receive this message, since the destination server of the message is *Server - D*.

DotNetMQ provides this functionality and simplicity. It finds the **best (shortest) path** from the source server to the destination server on the graph and forwards the message.

After this comprehensive introduction, letâ€™s see how to use DotNetMQ in practice.

Installing and Running DotNetMQ

There is no auto install for now, but it is very easy to install DotNetMQ. **download** and **unzip** the **binaries** download file from the top of the article. Just copy everything from there to *C:\Program Files\DotNetMQ* and run ***INSTALL_x86.bat*** *(or* ***INSTALL_x64.bat*** *if you are using 64bit operating system)*.

You can check Windows services to see if DotNetMQ is installed and working.

First Application Using DotNetMQ

Letâ€™s see DotNetMQ in action. To make the first application most simple, I assume that there are two console applications running on the same machine (in fact (as we will see later in this document) there is no significant difference if the applications are in different machines; the only difference is properly setting the name of the destination server in the message).

- **Application1**: Gets a string message from the user and sends it to Application2.
- **Application2**: Writes incoming messages to the console screen.

Registering Applications to DotNetMQ

We need to register applications once to use them with DotNetMQ. It is a very simple process. Run **DotNetMQ Manager** (*MDSManager.exe* in the DotNetMQ program folder (default: *C:\Program Files\DotNetMQ*)), and open **Application List** from the **Applications** menu. Click the **Add New Application** button and enter a name for the application.

Add the **Application1** and **Application2** applications to DotNetMQ as described above. Finally, your application list must be like below.

Figure - 5: Application list screen of the DotNetMQ Manager tool.

This screen shows the registered applications to DotNetMQ. The Connected Clients column shows the count of instances of the application that are currently connected to DotNetMQ. It is **not needed to restart** DotNetMQ because of the changes in this screen.

Developing Application1

Create a new console application with name **Application1** in Visual Studio and add a reference to **MDSCommonLib.dll** that provides necessary classes to connect to DotNetMQ. Then write the following code in the Program.cs file:

C# Shrink ▲

```
using System;
using System.Text;
using MDS.Client;

namespace Application1
{
```

```
    class Program
    {
        static void Main(string[] args)
        {
            //Create MDSClient object to connect to DotNetMQ
            //Name of this application: Application1
            var mdsClient = new MDSClient("Application1");

            //Connect to DotNetMQ server
            mdsClient.Connect();

            Console.WriteLine("Write a text and press enter to send " +
               "to Application2. Write 'exit' to stop application.");

            while (true)
            {
                //Get a message from user
                var messageText = Console.ReadLine();
                if (string.IsNullOrEmpty(messageText) || messageText == "exit")
                {
                    break;
                }

                //Create a DotNetMQ Message to send to Application2
                var message = mdsClient.CreateMessage();
                //Set destination application name
                message.DestinationApplicationName = "Application2";
                //Set message data
                message.MessageData = Encoding.UTF8.GetBytes(messageText);

                //Send message
                message.Send();
            }

            //Disconnect from DotNetMQ server
            mdsClient.Disconnect();
        }
    }
}
```

When creating the `MDSClient` object, we pass the application name which connects to DotNetMQ. With this constructor, we connect to DotNetMQ on the local server (127.0.0.1) with the default port number (10905). Overloaded constructors can be used to connect to another server and port.

The `CreateMessage` method of `MDSClient` returns an object of type `IOutgoingMessage`. The `MessageData` property is the actual data to send to the destination application. It is a byte array. We are converting the user input text to a byte array using UTF8 encoding.
The`DestinationApplicationName` and `DestinationServerName` properties are used to set the destination address of message. If we donâ€™t specify the destination server, it is assumed as the local server. Finally, we send the message.

Developing Application2

Create a new console application with name **Application2** in Visual Studio, add a reference to **MDSCommonLib.dll** and write the following code:

C# Shrink ▲

```
using System;
using System.Text;
using MDS.Client;

namespace Application2
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create MDSClient object to connect to DotNetMQ
            //Name of this application: Application2
            var mdsClient = new MDSClient("Application2");

            //Register to MessageReceived event to get messages.
            mdsClient.MessageReceived += MDSClient_MessageReceived;

            //Connect to DotNetMQ server
            mdsClient.Connect();

            //Wait user to press enter to terminate application
            Console.WriteLine("Press enter to exit...");
            Console.ReadLine();

            //Disconnect from DotNetMQ server
            mdsClient.Disconnect();
        }

        /// <summary>
        /// This method handles received messages from other applications via DotNetMQ.
        /// </summary>
        /// <param name="sender"></param>
        /// <param name="e">Message parameters</param>
        static void MDSClient_MessageReceived(object sender, MessageReceivedEventArgs e)
        {
            //Get message
            var messageText = Encoding.UTF8.GetString(e.Message.MessageData);

            //Process message
            Console.WriteLine();
            Console.WriteLine("Text message received : " + messageText);
            Console.WriteLine("Source application    : " +
e.Message.SourceApplicationName);

            //Acknowledge that message is properly handled
            //and processed. So, it will be deleted from queue.
            e.Message.Acknowledge();
        }
    }
}
```

Creating the `MDSClient` object is similar to that in Application1 but the application name is Application2. To receive messages for an application, it needs to register to the `MessageReceived` event of `MDSClient`. Then we connect to DotNetMQ and stay connected until the user presses Enter.

When a message is sent to Application2, the `MDSClient_MessageReceived` method handles the event. We get the message from the `Message` property of `MessageReceivedEventArgs`. The type of message is `IIncomingMessage`. The `MessageData` property of `IIncomingMessage` contains the actual message data that is sent by Application1. Since it is a byte array, we are converting it to string using UTF8 encoding. We write the message text that is sent by Application1 to the console screen.

Figure - 6: Application1 sends two messages to Application2 over DotNetMQ.

After processing an incoming message, it is needed to **Acknowledge** the message. That means the message is properly received and correctly processed. DotNetMQ then removes the message from message queue. We can also reject a message using the `Reject` method (if we can not process the message on an error case). In this situation, the message turns back to the message queue and will be

sent later to the destination application (or it will be sent to another instance of Application2 on the same server if exists). This is a powerful mechanism of the DotNetMQ system. Thus, it is guarantied that the message can not be lost and it is absolutely processed. If you do not acknowledge or reject a message, it is assumed as rejected. So, even if your application crashes, your message is sent back to your application later.

If you run **multiple instance** of the Application2, which one will receive messages? In this case, DotNetMQ delivers messages to applications sequentially. So, you can create multi sender/receiver systems. A message is received by only one instance of applications (applications receives different messages). DotNetMQ provides all functionallity and synchronization.

Transmit Rule Property of Message

Before sending a message, you can set the **Transmit Rule** of a message like this:

C#

```
message.TransmitRule = MessageTransmitRules.NonPersistent;
```

There are three types of transmit rules:

- `StoreAndForward`: This is the default transmit rule. Messages are persistent, can not be lost, and are guaranteed to be delivered. If the `Send` method does not throw an Exception, then the message is correctly received by DotNetMQ and stored in the database. It is stored in the database until the destination application receives and acknowledges it.
- `NonPersistent`**:** Messages are not stored in database. It is the **fastest** way of sending messages. A message is lost only if the DotNetMQ server stops.
- `DirectlySend`**:** This is an exclusive feature of DotNetMQ. This type of messages are directly sent to the application. The sender application is blocked until the receiver acknowledges a message. So, if the sender does not get any exception while calling the `Send` method, it means the message is properly received and acknowledged by the receiver application. If an error occurs while transmitting a message, the receiver is offline, or the receiver rejects the message, the sender gets an exception on the `Send` method. This rule correctly works even if applications are on different servers (even if there are many servers between applications).

Since the default transmit rule is `StoreAndForward`, letâ€™s try that:

- Run **Application1** (while Application2 is not running), write some messages, and close application.
- Run **Application2**, you will see that your messages are received by Application2 and are not lost.

Even if you stop the DotNetMQ service from Windows services after sending messages from Application1, your messages wonâ€™t be lost. That is called **persistence**.

CommunicationWay Property of MDSClient

By default, an application can send and receive messages using MDSClient (`CommunicationWays.SendAndReceive`). If an application doesnâ€™t want to receive messages, it must set the `CommunicationWay` property to `CommunicationWays.Send`. This property can be changed before connection or during communication with DotNetMQ.

ReConnectServerOnError Property of MDSClient

By default, MDSClient **automatically reconnects** to DotNetMQ if it disconnects. So, even if you restart DotNetMQ, it is not needed to restart your applications that are connected to DotNetMQ. You can set the `ReConnectServerOnError` property to `false` to disable auto-reconnect.

AutoAcknowledgeMessages Property of MDSClient

By default, You must **explicitly acknowledge** messages in **MessageReceived** event. Otherwise, it is assumed as **Rejected**. If you want oppisite of this approach, you must set **AutoAcknowledgeMessages** property as **true**. In this case, if your MessageReceived **event handler** does not throw an **exception** or you do not acknowledge/reject the message explicitly, it is automatically **acknowledged** (If an exception is thrown, the message is rejected).

Configuring DotNetMQ

You can configure DotNetMQ in two ways: Using **XML settings files** or **DotNetMQ Manager** (Windows Forms application). Here, I will show two approaches. Some of the configurations require you to **restart** DotNetMQ while others do not.

Servers

You may run DotNetMQ on only **one server**. In this situation, there is no need to configure anything for the servers. But if you want to run DotNetMQ on **more than one server** and make them communicate with others, you must define your **server graph**.

A server graph consists of two or more **nodes**. Each node is a server that has an **IP address** and **TCP port** (that is used by DotNetMQ). You can configure/**design** a server graph by using the DotNetMQ Manager.

Figure - 8: DotNetMQ Server Graph managing.

In the figure above, you see a server graph that consists of five nodes. The red node represents this server (this server means the server that you are connected with DotNetMQ Manager). A line means that there is a connection (and they can send/receive messages) between two nodes (they are called adjacent nodes). The name of the server/node in a graph is important and is used when sending messages to the server.

You can double-click a server in the graph to change its properties. To connect two servers, hold Ctrl, click the first one then the second one (to disconnect, do the same thing again). You can set a server as this server by right clicking and selecting **Set as this server**. You can also delete a server from the graph or add a new server by using the right click menu. Lastly, you can add move servers by dragging.

After designing your server graph, you must click the **Save & Update Graph** button to save the changes. The changes are saved to the *MDSSettings.xml* file in your DotNetMQ installation folder. You **must restart** DotNetMQ to apply the changes.

For the server graph above, the corresponding *MDSSettings.xml* settings are shown below:

XML Shrink ▲

```
<?xml version="1.0" encoding="utf-8"?>
<MDSConfiguration>
  <Settings>
    ...
  </Settings>
  <Servers>
    <Server Name="halil_pc" IpAddress="192.168.10.105"
       Port="10099" Adjacents="emre_pc" />
    <Server Name="emre_pc" IpAddress="192.168.10.244" Port="10099"
       Adjacents="halil_pc,out_server,webserver1,webserver2" />
    <Server Name="out_server" IpAddress="85.19.100.185"
       Port="10099" Adjacents="emre_pc" />
    <Server Name="webserver1" IpAddress="192.168.10.263"
       Port="10099" Adjacents="emre_pc,webserver2" />
    <Server Name="webserver2" IpAddress="192.168.10.44"
       Port="10099" Adjacents="emre_pc,webserver1" />
  </Servers>
  <Applications>
    ...
  </Applications>
  <Routes>
    ...
  </Routes>
</MDSConfiguration>
```

Surely, this configuration is made according to your real network. You must install DotNetMQ on all servers in the graph. Also, you must configure the same graph on all servers (you can easily copy the server nodes from the XML to the other servers).

DotNetMQ uses a **short path** algorithm to send the messages (if no manual route is defined in the settings file). Consider an **Application A** that is running on **halil_pc** and sending a message to **Application B** on **webserver2**. The path is simply: Application A -> **halil_pc -> emre_pc -> webserver2** -> Application B. halil_pc knows the next forwarding server (emre_pc) by using the server graph definition.

Lastly, the *MDSSettings.design.xml* file contains the server design information (locations of nodes on the screen). This is just needed in the server graph window in DotNetMQ Manager and not needed for the runtime of DotNetMQ.

Applications

As shown in Figure - 5, you can **add/remove applications** that are using DotNetMQ as a message broker. It is not needed to restart DotNetMQ for these changes. Application settings are also saved to the *MDSSettings.xml* file as shown below.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<MDSConfiguration>
```

```
  ...
  <Applications>
    <Application Name="Application1" />
    <Application Name="Application2" />
  </Applications>
  ...
</MDSConfiguration>
```

An application must be in this list to be able to connect to DotNetMQ. If you directly change the XML file, you must restart the DotNetMQ server.

Routing / Load Balancing

A usable feature of DotNetMQ is routing. Routing settings (for now) are configured only in the XML settings file (*MDSSettings.xml*). You can see two types of routing in the settings file below:

XML Shrink ▲

```
<?xml version="1.0" encoding="utf-8" ?>
<MDSConfiguration>
  ...
  <Routes>

    <Route Name="Route-App2" DistributionType="Sequential" >
      <Filters>
        <Filter DestinationServer="this" DestinationApplication="Application1" />
      </Filters>
      <Destinations>
        <Destination Server="Server-A" Application="Application1" RouteFactor="1" />
        <Destination Server="Server-B" Application="Application1" RouteFactor="1" />
        <Destination Server="Server-C" Application="Application1" RouteFactor="1" />
    </Destinations>
    </Route>

    <Route Name="Route-App2" DistributionType="Random" >
      <Filters>
        <Filter DestinationServer="this" DestinationApplication="Application2" />
        <Filter SourceApplication="Application2" TransmitRule="StoreAndForward" />
    </Filters>
      <Destinations>
        <Destination Server="Server-A" Application="Application2" RouteFactor="1" />
        <Destination Server="Server-B" Application="Application2" RouteFactor="3" />
      </Destinations>
    </Route>

  </Routes>
  ...
</MDSConfiguration>
```

A `Route` node has two attribute: `Name` is a user-friendly name of the Route entry (does not affect routing) and `DistributionType` is the strategy of the routing. There are two types of routing strategies:

- `Sequential`: Messages are routed to destination servers sequentially. The `RouteFactor` of destinations are considered while distributing.
- `Random`: Messages are routed to destination servers randomly. Probability of selecting the server A is: (RouteFactor(A) / (Total of all RouteFactor values of all destinations in the route definition)).

Filters are used to decide which route to use for a message. If properties of a message are suitable for one of the filters, the message is routed. There are **five conditions** (XML attributes) to define a filter:

- `SourceServer`: The first source server of the message. Can be `this` to indicate this server.
- `SourceApplication`: Sender application of the message.
- `DestinationServer`: Last destination server of the message. Can be `this` to indicate this server.
- `DestinationApplication`: The application that will receive the message.
- `TransmitRule`: One of these transmit rules: `StoreAndForward`, `DirectlySend`, or `NonPersistent`.

If one or more condition is not declared, it is not considered while filtering messages. So, if all the conditions are empty (or not declared), all messages are fit to this filter. A filter is selected for a message only if all the conditions are fit to the message. If a message is proper for (at least) one of the filters of a route, the route is selected and used.

Destinations are used to route messages to other servers. One of the destinations is selected according to the `DistributionType` property of the `Route` entry (explained before). A destination **must define three attributes**:

- `Server`: Destination server. Can be `this` to indicate this server.
- `Application`: Destination application. Destination application is generally defined as same as the original destination, but you can **redirect** a message to another application than its original destination application.
- `RouteFactor`: This property is used to indicate the relative selection ratio of a destination. The `RouteFactor` attribute can be used for **load balancing**. If you want to distribute messages to all servers equally, you can define this as 1 for all destinations. But if you have two servers and one of them is more powerful than other, you can select the first server twice more than the second one by defining the appropriate route factors.

You **must restart** DotNetMQ after changing routes.

Other Settings

DotNetMQ currently supports three **storage** types: **SQLite** (default), **MySQL**, and **Memory**. You can change the storage type in the *MDSSettings.xml* file.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<MDSConfiguration>
  ...
  <Settings>
    <Setting Key="ThisServerName" Value="halil_pc" />
```

```
    <Setting Key="StorageType" Value="SQLite" />
  </Settings>
  ...
</MDSConfiguration>
```

The storage types must be one of the following values:

- **SQLite**: Uses the SQLite database system. This is the default storage type. Uses the *{DotNetMQ-Install-Directory}\SqliteDB\MDS.s3db* file as the database.
- **MSSQL**: Uses a Microsoft SQL Server database. You must supply the `ConnectionString` setting as the connection string (will be explained).
- **MySQL-ODBC**: Uses a MySQL database with ODBC. You must supply the `ConnectionString` setting as the connection string.
- **MySQL-Net**: Uses a MySQL database with a **.NET Adapter**. You must supply the `ConnectionString` setting as connection string.
- **Memory**: Uses memory as the storage device. In this case, persistent messages are lost if DotNetMQ is stopped.

Here is a sample configuration to use the **MySQL-ODBC** storage type:

XML

```
<Settings>
    <Setting Key="ThisServerName" Value="halil_pc" />
    <Setting Key="StorageType" Value="MySQL-ODBC" />
    <Setting Key="ConnectionString"
       Value="uid=root;server=localhost;driver={MySQL ODBC 3.51 Driver};database=mds" />
  </Settings>
```

You can find needed files in the ***Setup\Databases*** folder (in the DotNetMQ installation folder) that are needed to create database and tables that are used by DotNetMQ. Feel free to ask questions to me if you have a problem.

There is also another setting to define the name of the current/this server (`ThisServerName`). It must be one of the servers in the `Servers` section. If you use the DotNetMQ Manager to edit your servers graph, it is automatically set.

Messaging Over Network

Sending a message to an application on a remote server is easy as sending a message to an application on the current server.

A Simple Application

Let's consider the network below.

Figure - 8: Messaging of two applications over network with DotNetMQ.

There is an application (Application1) running on ServerA that wants to send a message to another application (Application2) on ServerC and there is no direct connection between ServerA and ServerC because of the firewall rules. Let's change the applications we developed in the **First Applications** section.

There is not even a single change in Application2. Just run Application2 in ServerC and wait for incoming messages.

There is a minor change in Application1 on how we send a message. It must set the `DestinationServerName` of the message as **ServerC**.

JavaScript

```
var message = mdsClient.CreateMessage();
message.DestinationServerName = "ServerC"; //Set destination server name here!
message.DestinationApplicationName = "Application2";
message.MessageData = Encoding.UTF8.GetBytes(messageText);
message.Send();
```

That's all. You do not have to know where ServerC is, for a direct connection to ServerC... They are all defined in the DotNetMQ settings. Note that if you do not set the `DestinationServerName` of a message, it is assumed as **current/this** server and DotNetMQ sends the message to the application on the same server. Also, if you define the necessary routings, you don't have to set the destination server: it is routed by DotNetMQ automatically.

Surely, DotNetMQ settings must be properly set according to the server connections (server graph), and Application1 and Application2 must be **registered** to the DotNetMQ server as described in the **Configuring DotNetMQ** section.

A Real Life Case: **Distributed SMS Processor**

As you already saw, DotNetMQ can be used to build **distributed**, **load balanced** application systems. In this section, I'll discuss a real life scenario: A distributed SMS process system.

Assume that there is a short message (SMS) service that is used for polling a music competition. After all competitors sing their songs, audience send messages like "VOTE 103" to our SMS service to vote for their favourite competitor (103 is a sample code to vote for a specific competitor). And assume that this polling is done in just 30 minutes and approximately five million people will send SMSs to our service.

We will receive every message, process it (parse the SMS text, update the database to increase the vote count of the competitor) and send a confirmation message to the sender of the SMS. We must receive messages from two servers, process messages on four servers, and send confirmation messages from two servers. We have totally eight servers. Let's see our complete system diagram:

Figure - 9: A distributed SMS processing system

There are three types of applications: **Receiver**, **Processor**, and **Sender**. You can use DotNetMQ as the message queue and load balancer in such a scenario to build a distributed, scalable message processing system by configuring the server graph and routes as described in the **Configuring DotNetMQ** section.

Request/Reply Style Messaging with DotNetMQ

In most cases, an application sends a message to another application and gets a response message. DotNetMQ has **built-in support** for this type of messaging. Think about a service that is used to query a stock status. There are two types of messages:

C#

```
[Serializable]
public class StockQueryMessage
{
    public string StockCode { get; set; }
}

[Serializable]
public class StockQueryResultMessage
{
    public string StockCode { get; set; }
    public int ReservedStockCount { get; set; }
    public int TotalStockCount { get; set; }
}
```

A simple **Stock server** code is shown below.

C# Shrink ▲

```
using System;
using MDS;
using MDS.Client;
using StockCommonLib;

namespace StockServer
{
    class Program
    {
        static void Main(string[] args)
        {
            var mdsClient = new MDSClient("StockServer");
            mdsClient.MessageReceived += MDSClient_MessageReceived;

            mdsClient.Connect();

            Console.WriteLine("Press enter to exit...");
            Console.ReadLine();

            mdsClient.Disconnect();
        }

        static void MDSClient_MessageReceived(object sender,
                    MessageReceivedEventArgs e)
        {
            //Get message
            var stockQueryMessage =
                GeneralHelper.DeserializeObject(e.Message.MessageData)
                as StockQueryMessage;
            if (stockQueryMessage == null)
            {
                return;
            }

            //Write message content
            Console.WriteLine("Stock Query Message for: " +
                              stockQueryMessage.StockCode);
```

```
            //Get stock counts from a database...
            int reservedStockCount;
            int totalStockCount;
            switch (stockQueryMessage.StockCode)
            {
                case "S01":
                    reservedStockCount = 14;
                    totalStockCount = 80;
                    break;
                case "S02":
                    reservedStockCount = 0;
                    totalStockCount = 25;
                    break;
                default: //Stock does not exists!
                    reservedStockCount = -1;
                    totalStockCount = -1;
                    break;
            }

            //Create a reply message for stock query
            var stockQueryResult = new StockQueryResultMessage
                                       {
                                           StockCode = stockQueryMessage.StockCode,
                                           ReservedStockCount = reservedStockCount,
                                           TotalStockCount = totalStockCount
                                       };

            //Create a MDS response message to send to client
            var responseMessage = e.Message.CreateResponseMessage();
            responseMessage.MessageData =
               GeneralHelper.SerializeObject(stockQueryResult);

            //Send message
            responseMessage.Send();

            //Acknowledge the original request message.
            //So, it will be deleted from queue.
            e.Message.Acknowledge();
        }
    }
}
```

The stock server listens for incoming `StockQueryMessage` objects and sends a `StockQueryResultMessage` to the sender. To be simple, I did not select stocks from a database. A response message is created by the `CreateResponseMessage()` method of the incoming message. Lastly, the message is acknowledged after a response is sent. Now I will show a simple stock client code to get stock information from a server:

C# Shrink ▲

```
using System;
using MDS;
using MDS.Client;
using MDS.Communication.Messages;
using StockCommonLib;
```

```
namespace StockApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Press enter to query a stock status");
            Console.ReadLine();

            //Connect to DotNetMQ
            var mdsClient = new MDSClient("StockClient");
            mdsClient.MessageReceived += mdsClient_MessageReceived;
            mdsClient.Connect();

            //Create a stock request message
            var stockQueryMessage = new StockQueryMessage { StockCode = "S01" };

            //Create a MDS message
            var requestMessage = mdsClient.CreateMessage();
            requestMessage.DestinationApplicationName = "StockServer";
            requestMessage.TransmitRule = MessageTransmitRules.NonPersistent;
            requestMessage.MessageData = GeneralHelper.SerializeObject(stockQueryMessage);

            //Send message and get response
            var responseMessage = requestMessage.SendAndGetResponse();

            //Get stock query result message from response message
            var stockResult = (StockQueryResultMessage)
              GeneralHelper.DeserializeObject(responseMessage.MessageData);

            //Write stock query result
            Console.WriteLine("StockCode          = " + stockResult.StockCode);
            Console.WriteLine("ReservedStockCount = " + stockResult.ReservedStockCount);
            Console.WriteLine("TotalStockCount    = " + stockResult.TotalStockCount);

            //Acknowledge received message
            responseMessage.Acknowledge();

            Console.ReadLine();

            //Disconnect from DotNetMQ server.
            mdsClient.Disconnect();
        }

        static void mdsClient_MessageReceived(object sender,
                    MessageReceivedEventArgs e)
        {
            //Simply acknowledge other received messages
            e.Message.Acknowledge();
        }
    }
}
```

In the sample above, `TransmitRule` is selected as `NonPersistent` to show a sample usage. Surely, you can send `StoreAndForward` (persistent) messages. Here is a sample screenshot of the running

applications:

Figure - 10: Request/Reply style messaging applications.

Service-Oriented Architecture in DotNetMQ

SOA (Service-Oriented Architecture) has been a popular concept for many years. Web Services and WCF are two major solutions to SOA. Generally, it is not expected to support SOA from a Message Queue system. Also messaging is an asynchronous and loosely coupled process while a Web Service method call is typically synchronous and tight coupled. Even (as you saw in the previous sample applications) messaging is not as easy as calling a remote method. But when your message count increases, your application becomes complicated and harder to maintain.

DotNetMQ supports remote method invocation mechanism upon persistent or non-persistent messages. So you can call a remote method asynchronously that is guaranteed to be called and guaranteed to be succeed!

Sample Application: **SMS/Mail Sender**

Here we will develop a simple service that can be used to send SMS and e-mail. Maybe it is not needed to write a service for sending an email/SMS, all applications can do it themselves. But imagine that you have many applications that are sending emails. What if the mail server has a problem while sending an email? The application must try until it successfully sends the email. So you must build a queue mechanism in your application to try and send the email again and again. At the worse case, your application may be a short time running application (such as a Web Service) or must be closed

before sending the email. But you have to send the email when the mail servers come online and the mail must not be lost.

In this case, you can develop a separate mail/SMS service that will try to send the SMS/mail until it is successfully sent. You can develop a mail service that receives mail requests over DotNetMQ and acknowledge requests (messages) only if the email is successfully sent. If sending fails, just do not acknowledge (or reject) the message, thus it will be tried again later.

Service

We will first develop the mail/SMS service. To do this, we must define a class that is delivered from the `MDSService` base class:

C# Shrink ▲

```
using System;
using MDS.Client.MDSServices;

namespace SmsMailServer
{
    [MDSService(Description = "This service is a " +
              "sample mail/sms service.", Version = "1.0.0.0")]
    public class MyMailSmsService : MDSService
    {
        //All parameters and return values can be defined.
        [MDSServiceMethod(Description = "This method is used send an SMS.")]
        public void SendSms(
            [MDSServiceMethodParameter("Phone number to send SMS.")] string phone,
            [MDSServiceMethodParameter("SMS text to be sent.")] string smsText)
        {
            //Process SMS
            Console.WriteLine("Sending SMS to phone: " + phone);
            Console.WriteLine("Sms Text: " + smsText);

            //Acknowledge the message
            IncomingMessage.Acknowledge();
        }

        //You do not have to define any parameters
        [MDSServiceMethod]
        public void SendEmail(string emailAddress, string header, string body)
        {
            //Process email
            Console.WriteLine("Sending an email to " + emailAddress);
            Console.WriteLine("Header: " + header);
            Console.WriteLine("Body  : " + body);

            //Acknowledge the message
            IncomingMessage.Acknowledge();
        }

        // A simple method just to show return values.
        [MDSServiceMethod]
```

```
        [return: MDSServiceMethodParameter("True, if phone number is valid.")]
        public bool IsValidPhone([MDSServiceMethodParameter(
                "Phone number to send SMS.")] string phone)
        {
            //Acknowledge the message
            IncomingMessage.Acknowledge();

            //Return result
            return (phone.Length == 10);
        }
    }
}
```

As you can see, it is just a regular C# class decorated with attributes. The `MDSService` and `MDSServiceMethod` attributes must be defined and all other attributes are optional (but it is good to write them). We will see soon why they are used). Your service methods must have the `MDSServiceMethod` attribute. If you do not want to expose some of your methods, simply do not add the `MDSServiceMethod` attribute.

We must also **acknowledge** the message in the service method. Otherwise, the message (that caused this method call) will not be deleted from the message queue and our method will be called again. We can also **reject** the message if we can not process it (for example, if the mail server is not working and we can not send emails). If we reject the message, it will be sent to us later (**reliability**). You can reach the original message using the `IncomingMessage` property of the `MDSService` class. Also, you can get a remote application's (that called the service method) information using the `RemoteApplication` property.

After creating a proper service class, we must create an application to run it. Here is a simple console application that runs our `MyMailSmsService`:

C#

```
using System;
using MDS.Client.MDSServices;

namespace SmsMailServer
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var service =
                      new MDSServiceApplication("MyMailSmsService"))
            {
                service.AddService(new MyMailSmsService());
                service.Connect();

                Console.WriteLine("Press any key to stop service");
                Console.ReadLine();
            }
        }
```

```
    }
}
```

As you can see, it is just three lines of code to create and run a service. Since `MDSService` is disposable, you can use a `using` statement. Also, you can close the service manually with the `Disconnect` method of `MDSServiceApplication`. You can run more than one service on a single `MDSServiceApplication` using the `AddService` method.

Client

To develop an application that uses a DotNetMQ service, you must create a **service proxy** (like Web Services and WCF). To do this, you can use the **MDSServiceProxyGenerator** tool. First, compile your service project, than run *MDSServiceProxyGenerator.exe* (in the DotNetMQ installation folder).

Figure - 11: Generating a proxy class for a service in DotNetMQ.

Select your service **assembly** file (*SmsMailServer.exe* in this sample project). You can select the **service class** or generate proxies of all services in a given assembly. Enter a **namespace** and the **target folder** to **generate the proxy class**. After generating the proxy class, you can **add** it to your project.

I will not show the internals of this proxy class and you do have to know it (you can see it in the source code, it is a very simple class). Your method/parameter **attributes** are used to generate the **code comments** in this proxy file.

After adding the generated proxy class to our project, we can simply send messages to the service just like **simple method calls**:

C# Shrink ▲

```
using System;
using MDS.Client;
using MDS.Client.MDSServices;
using SampleService;

namespace SmsMailClient
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Press enter to test SendSms method");
            Console.ReadLine();

            //Application3 is name of an application that sends sms/email.
            using (var serviceConsumer = new MDSServiceConsumer("Application3"))
            {
                //Connect to DotNetMQ server
                serviceConsumer.Connect();

                //Create service proxy to call remote methods
                var service = new MyMailSmsServiceProxy(serviceConsumer,
                    new MDSRemoteAppEndPoint("MyMailSmsService"));

                //Call SendSms method
                service.SendSms("3221234567", "Hello service!");
            }
        }
    }
}
```

You can also call other methods of the service, and get return values as regular method calls. Actually, your method calls are translated to reliable messages. For instance, even if the remote application (`MyMailSmsService`) is not running when `SendSms` is called, it is called when the service starts to run, so your method calls are also guarantied to be called.

You can change the transmit rule for messaging using the `TransmitRule` property of the service proxy. If a service method returns `void`, its transmit rule is `StoreAndForward` by default. If a service method

returns a value, the method call can not be made reliable (since the method call is synchronous and waiting a result), it's rule is `DirectlySend`.

You can choose any type as method parameters. If it is a primary type (`string`, `int`, `byte`...) there is no need for additional settings but if you want to use your own classes as a method parameter, the class must be marked as `Serializable` since DotNetMQ uses binary serialization for parameters.

Note that you must **register MyMailSmsService** and **Application3** to DotNetMQ before running this sample.

Web Services Support

Surely, you can connect to DotNetMQ in a Web Service since it is also a .NET application. But, what if you want to write an ASP.NET Web method to handle messages for an application (and can reply with a message in the same context)? Web Services are suitable for such Request/Reply style method calls.

DotNetMQ supports ASP.NET web services and can deliver messages to Web Services. There is a template web service in samples (in download files) to accomplish that. It is defined as below:

C# Shrink ▲

```
using System;
using System.Web.Services;
using MDS.Client.WebServices;

[WebService(Namespace = "http://www.dotnetmq.com/mds")]
[WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]
public class MDSAppService : WebService
{
    /// <summary>
    /// MDS server sends messages to this method.
    /// </summary>
    /// <param name="bytesOfMessage">Byte array form of message</param>
    /// <returns>Response message to incoming message</returns>
    [WebMethod(Description = "Receives incoming messages to this web service.")]
    public byte[] ReceiveMDSMessage(byte[] bytesOfMessage)
    {
        var message = WebServiceHelper.DeserializeMessage(bytesOfMessage);
        try
        {
            var response = ProcessMDSMessage(message);
            return WebServiceHelper.SerializeMessage(response);
        }
        catch (Exception ex)
        {
            var response = message.CreateResponseMessage();
            response.Result.Success = false;
            response.Result.ResultText =
              "Error in ProcessMDSMessage method: " + ex.Message;
            return WebServiceHelper.SerializeMessage(response);
        }
```

```
    }

    /// <summary>
    /// Processes incoming messages to this web service.
    /// </summary>
    /// <param name="message">Message to process</param>
    /// <returns>Response Message</returns>
    private IWebServiceResponseMessage
            ProcessMDSMessage(IWebServiceIncomingMessage message)
    {
        //Process message

        //Send response/result
        var response = message.CreateResponseMessage();
        response.Result.Success = true;
        return response;
    }
}
```

You do not change the `ReceiveMDSMessage` method and must process the message in the `ProcessMDSMessage` method as shown above. Also, you must define the **address** of your Web Service in *MDSSettings.xml* as shown below. You can also add Web Services using the DotNetMQ management tool.

XML

```
...
<Applications>
  <Application Name="SampleWebServiceApp">
    <Communication Type="WebService"
      Url="http://localhost/SampleWebApplication/SampleService.asmx" />
  </Application>
</Applications>
...
```

Performance of DotNetMQ

There are some test results for messaging in DotNetMQ:

Messaging:

- **10,000** messages in ~**25** seconds as **persistent** (~**400** messages/second).
- **10,000** messages in ~**3.5** seconds as **non-persistent** (~**2,850** messages/second).

Method Calls (in DotNetMQ Services)

- **10,000** method calls in ~**25** seconds as **persistent** (~**400** calls/second).
- **10,000** method calls in ~**8.7** seconds as **non-persistent** (~**1,150** calls/second).

Test Platform: Intel Core 2 Duo 3,00 GHz CPU. 2 GB RAM PC. Messages/calls are made between two applications running on the same computer.

References

- [1] Book: **Enterprise Integration Patterns**: Designing, Building, and Deploying Messaging Solutions by Gregor Hohpe, Bobby Woolf (Addison Wesley, 2003).

History

- **23.05.2011** (DotNetMQ v0.9.1.0)
  - Added Microsoft SQL Server database support.
  - Changed `MySQLConnectionString` setting to `ConnectionString`.
  - Source code updated.
  - Article updated according to changes.
- **16.05.2011** (DotNetMQ v0.9.0.0)
  - Added sample Web Service template to downloads.
  - Some fixes and additions to the article.
- **09.05.2011** (DotNetMQ v0.9.0.0)
  - First published.

License

This article, along with any associated source code and files, is licensed under The GNU Lesser General Public License (LGPLv3)

Written By

Halil ibrahim Kalkan

Founder Volosoft

Turkey

I have started programming at 14 years old using Pascal as hobby. Then I interested in web development (HTML, JavaScript, ASP...) before university.

I graduated from Sakarya University Computer Engineering. At university, I learned C++, Visual Basic.NET, C#, ASP.NET and Java. I partly implemented ARP, IP and TCP protocols in Java as my final term project.

Now, I am working on Windows and web based software development mostly using Microsoft technologies in my own company.

My open source projects:

* ASP.NET Boilerplate: http://aspnetboilerplate.com
* jTable: http://jtable.org
* Others: https://github.com/hikalkan

My personal web site:

http://www.halilibrahimkalkan.com

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

Excellent article - How to make it work with SqLite?
Member 14872434 9-Oct-21 6:44

Nice one which is similar to RabbitMQ
Member 12109411 16-Oct-19 6:01

My vote of 5
Wooters 30-Sep-18 8:16

OMG
george.kitia 10-Aug-18 10:11

Just awesome - full viable solution for distributed queues
mtlabs 10-Jul-18 21:16

Amazing & Incredible Solution

Member 12002836 4-Jul-18 23:53

The service is not responding to the control function
Member 12161531 11-Jan-18 18:47

Windows Services Installation for DotNetMQ under win7 OS
Member 12835252 10-Nov-16 19:59

Re: Windows Services Installation for DotNetMQ under win7 OS
karpediemnow 13-Apr-17 0:59

DotnetMQ services still can't start
Member 12783478 8-Oct-16 17:48

MessageQueue Features
Member 12093789 21-Apr-16 4:42

Connect between 2 applications on 2 servers
Member 7716847 28-Mar-16 8:23

Client's state is not connected. It can not send message
Jaimesh.2411 25-Mar-16 2:47

Re: Client's state is not connected. It can not send message
Member 7716847 26-Mar-16 6:45

Re: Client's state is not connected. It can not send message
Jaimesh.2411 27-Mar-16 19:46

Re: Client's state is not connected. It can not send message
sylfree9999 8-Oct-17 18:06

Unable to install DOTNETMQ
Ananth Asamani 12-Mar-16 2:50

Re: Unable to install DOTNETMQ
Member 7716847 26-Mar-16 6:44

Re: Unable to install DOTNETMQ
qqjxzmm 14-Jun-16 21:49

Re: Unable to install DOTNETMQ
Member 12783478 8-Oct-16 17:52

how to commnicate with Android App Client?
maosl1985 20-Jan-16 18:10

Reading messages instead of event raise
andrear 3-Dec-15 8:10

Sending message from Linux
mickey3D 24-Nov-15 18:11

Sequential Order of Messages
carloagl 6-Oct-15 3:06

Excellent Article
UpdateAk47 19-Sep-15 11:21

Refresh 1 2 3 4 5 6 7 8 9 10 11 Next ▷

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2011 by Halil ibrahim Kalkan
Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1